

MD

BANKING MANAGER

Projektdokumentation zum Web- und Datenbankenpraktikum

Milad Orifien und Dhruvit Goti

Betreuer: Sebastian Greiner

Projektzeitraum: 29.06.2026 bis 27.07.2026

Sommersemester 2026

1. Projektüberblick

1.1 Auswahl des Projekts

Für das Abschlussprojekt standen drei verschiedene Themen zur Auswahl. Wir haben uns für den Banking Manager entschieden, weil dieses Thema für uns am interessantesten war und aus unserer Sicht die meisten Möglichkeiten für eine abwechslungsreiche Webanwendung geboten hat.

Der Name MD Banking Manager setzt sich aus unseren Vornamen Milad und Dhruvit sowie dem Projektthema Banking Manager zusammen.

1.2 Ziel des Projekts

Unser Ziel war es, eine stabile und übersichtliche Anwendung zu entwickeln, die alle geforderten Funktionen zuverlässig erfüllt. Gleichzeitig sollte der Code so aufgebaut sein, dass wir ihn auch nach dem Projekt noch verstehen, warten und bei Bedarf erweitern können.

Uns war außerdem wichtig, kein reines Abgabeprojekt zu erstellen. Der MD Banking Manager sollte einen Stand erreichen, den wir später als eigenes Projekt in unser Portfolio aufnehmen und beispielsweise bei Bewerbungen oder im Lebenslauf erwähnen können. Neben der technischen Funktionalität haben wir deshalb auch auf eine verständliche Ordnerstruktur, ein einheitliches Design und eine möglichst einfache Bedienung geachtet.

1.3 Verwendete Technologien

Technologie	Verwendung im Projekt
Python	Programmlogik und Verarbeitung
Flask	Webframework und Routen
PostgreSQL	Relationale Datenbank
HTML und Jinja	Aufbau und dynamische Ausgabe der Seiten
CSS	Gestaltung der Benutzeroberfläche
JavaScript	Kleinere interaktive Funktionen
OpenPyXL	Import und Export von Excel-Dateien
python-dotenv	Einlesen lokaler Konfigurationseinstellungen
Git und GitHub	Sicherung des finalen Projektstands

2. Funktionen des MD Banking Managers

Der MD Banking Manager enthält neben den vorgegebenen Grundfunktionen auch einige zusätzliche Funktionen. Uns war dabei wichtig, dass die Anwendung trotz des größeren Umfangs übersichtlich bleibt.

2.1 Registrierung und Anmeldung

Neue Nutzer können sich mit Name, E-Mail-Adresse und Passwort registrieren und anschließend anmelden. Nach der Anmeldung gelangen sie auf ihre persönliche Startseite. Über das Profil lassen sich persönliche Angaben und das Passwort später ändern.

2.2 Finanzkonten

Jeder Nutzer kann mehrere eigene Finanzkonten erstellen, individuell benennen, umbenennen und löschen. Es handelt sich nicht um echte verbundene Bankkonten, sondern um Konten, die innerhalb der Anwendung selbst verwaltet werden. Der Kontostand ergibt sich aus der Summe aller zugehörigen Einnahmen und Ausgaben.

2.3 Buchungen

Für jedes Finanzkonto können Buchungen mit Datum, Uhrzeit, Betrag, Empfänger, Verwendungszweck und Kategorie erfasst werden. Positive Beträge werden als Einnahmen und negative Beträge als Ausgaben behandelt. Buchungen können gesucht, nach Zeitraum und Betrag gefiltert sowie nach mehreren Spalten sortiert werden.

2.4 Kategorien und Schlagwörter

Nutzer können eigene Kategorien und passende Schlagwörter anlegen. Beim Erstellen oder Importieren einer Buchung werden Empfänger und Verwendungszweck mit den vorhandenen Schlagwörtern verglichen. Wird kein passendes Schlagwort gefunden, wird intern die Systemkategorie „Nicht kategorisiert“ verwendet und in der Oberfläche als „Ohne Kategorie“ angezeigt.

2.5 Excel-Import und -Export

Buchungen können aus einer Excel-Datei importiert und für ein Finanzkonto wieder als Excel-Datei exportiert werden. Eine Beispieldatei im Projekt zeigt den benötigten Aufbau. Auch beim Import wird die automatische Kategoriezuordnung angewendet.

2.6 Auswertungen

Ausgaben werden nach Kategorien zusammengefasst. Die Auswertung kann für den gesamten Zeitraum, einen Monat, ein Jahr oder einen frei gewählten Zeitraum erstellt werden. Zusätzlich kann zwischen einem einzelnen Finanzkonto und allen eigenen Finanzkonten gewählt werden.

2.7 Gruppen und Nachrichten

Nutzer können Gruppen erstellen, andere registrierte Nutzer einladen und die Finanzkonten anderer Gruppenmitglieder lesend ansehen. Fremde Finanzkonten und Buchungen können nicht bearbeitet oder gelöscht werden. Zusätzlich können registrierte Nutzer private Nachrichten innerhalb der Anwendung versenden.

3. Technischer Aufbau

3.1 Schichtenarchitektur

Beim Aufbau des Projekts haben wir die einzelnen Aufgabenbereiche voneinander getrennt. Die Routes nehmen Anfragen aus dem Browser entgegen, die Services verarbeiten die Logik und die Repositories führen die Datenbankabfragen aus. Dadurch befinden sich HTML-Darstellung, Verarbeitung und SQL-Abfragen nicht vermisch in einer einzigen Datei.

Ablauf einer Anfrage

Browser → Routes → Services → Repositories → PostgreSQL

Die Rückgabe läuft anschließend in umgekehrter Richtung zurück zur HTML-Seite.

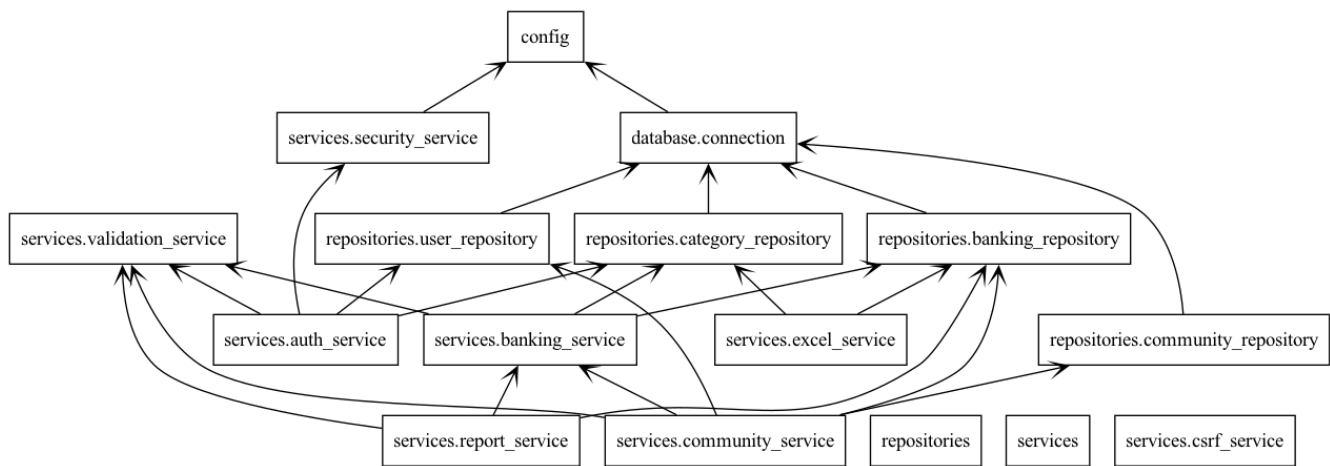


Abbildung 1: Paketdiagramm

3.2 Ordnerstruktur

Ordner / Datei	Aufgabe
routes/	Flask-Routen und Verarbeitung von Browseranfragen
services/	Logik, Validierung, Sicherheit, Excel und Auswertungen
repositories/	Parametrisierte SQL-Abfragen
templates/	HTML-Seiten mit Jinja
static/	CSS und JavaScript
database/	Datenbankverbindung, Schema und SQL-Dump
examples/	Beispieldatei für den Excel-Import
app.py	Start und Grundkonfiguration der Flask-Anwendung
config.py	Einlesen der Konfiguration
init_db.py	Neuanlage der Tabellen und Demodaten

3.3 Datenbankstruktur

Die wichtigsten Tabellen speichern Nutzer und Passwörter, Finanzkonten, Buchungen, Kategorien und Schlagwörter sowie Gruppen, Mitgliedschaften, Einladungen und Nachrichten. Ein Nutzer kann mehrere Finanzkonten besitzen, jedes Finanzkonto mehrere Buchungen. Fremdschlüssel sichern die wichtigsten Beziehungen direkt in PostgreSQL ab.

3.5 Klassen und UML-Diagramm

Klasse	Verantwortliche Person
Database	Milad Orifien
BankingRepository	Milad Orifien

Klasse	Verantwortliche Person
CategoryRepository	Milad Orifien
BankingService	Milad Orifien
ExcelService	Milad Orifien
ReportService	Milad Orifien
ValidationService	Milad Orifien
Config	Dhruvit Goti
UserRepository	Dhruvit Goti
CommunityRepository	Dhruvit Goti
AuthService	Dhruvit Goti
CommunityService	Dhruvit Goti
SecurityService	Dhruvit Goti
CsrfService	Dhruvit Goti

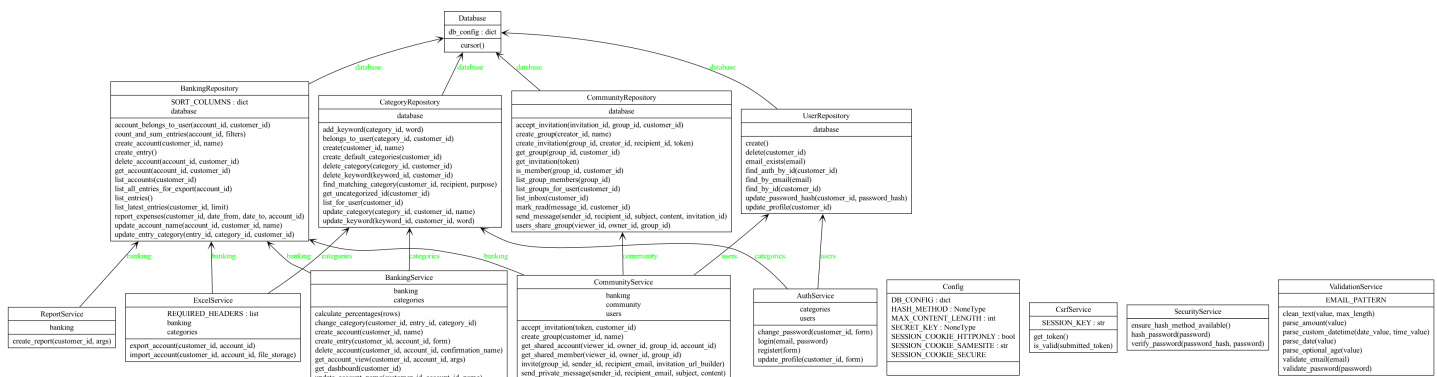


Abbildung 2: UML-Klassendiagramm

Das automatisch erzeugte Klassendiagramm zeigt die zentralen Klassen der Repository- und Service-Schicht. Die zusätzlich in der Verantwortlichkeitsübersicht aufgeführten Klassen „Database“ und „Config“ gehören ebenfalls zum Projekt, waren jedoch nicht Bestandteil der für Pyreverse ausgewählten Ordner.

3.5 Konfiguration über die .env-Datei

Lokale Einstellungen werden aus einer .env-Datei eingelesen. Dadurch können Zugangsdaten und Ports geändert werden, ohne den Programmcode anzupassen. Die echte .env-Datei wird nicht abgegeben. Im Projekt liegt nur eine .env.example mit Platzhaltern.

Variable	Bedeutung
APP_SECRET_KEY	Geheimer Flask-Schlüssel zur Signierung der Sitzung und zum Schutz vor manipulierten Session-Cookies. Er wird auch im

Variable	Bedeutung
	Zusammenhang mit dem CSRF-Schutz benötigt.
DB_HOST	Rechner, auf dem PostgreSQL läuft; lokal wird localhost verwendet.
DB_PORT	Gewählter PostgreSQL-Port. Im Projekt wird bewusst Port 6767 verwendet.
DB_NAME	Name der verwendeten PostgreSQL-Datenbank.
DB_USER	Benutzername für die Datenbankanmeldung.
DB_PASSWORD	Lokales PostgreSQL-Passwort.
FLASK_PORT	Port der Webanwendung. Verwendet wird 5001, da Port 5000 auf dem Entwicklungs-Mac bereits durch einen macOS-Dienst belegt war.
FLASK_DEBUG	Legt fest, ob der Flask-Debugmodus aktiviert ist. Für den normalen Start ist er ausgeschaltet.

4. Sicherheit und technische Entscheidungen

4.1 Passwortspeicherung

Passwörter werden nicht im Klartext gespeichert. Stattdessen wird PBKDF2 mit SHA-256 verwendet. Beim Anmelden wird das eingegebene Passwort mit dem gespeicherten Hash verglichen.

4.2 Parametrisierte SQL-Abfragen

Benutzereingaben werden über Platzhalter getrennt vom eigentlichen SQL-Befehl an PostgreSQL übergeben. Dadurch können Eingaben nicht einfach als zusätzlicher SQL-Code ausgeführt werden. Sortierspalten werden über eine feste Liste erlaubter Werte ausgewählt.

4.3 render_template() und Jinja

Die HTML-Seiten werden mit Flasks render_template() und Jinja erzeugt. Jinja maskiert dynamische Inhalte standardmäßig.

4.4 Zugriffsprüfungen

Geschützte Seiten setzen eine gültige Anmeldung voraus. Beim Aufruf eines Finanzkontos wird zusätzlich geprüft, ob es dem angemeldeten Nutzer gehört. Geteilte Gruppenansichten sind nur für Gruppenmitglieder und ausschließlich lesend erreichbar.

4.5 CSRF-Schutz

Formulare, die Daten verändern, enthalten einen zufälligen Token. Dieser Token wird beim Absenden mit dem Wert der aktuellen Sitzung verglichen. Eine Anfrage mit fehlendem oder falschem Token wird abgelehnt.

4.6 Eingabevalidierung

E-Mail-Adressen, Passwörter, Geldbeträge, Datum, Uhrzeit und Textlängen werden vor dem Speichern geprüft. Ungültige Eingaben führen zu einer verständlichen Fehlermeldung.

4.7 Systemkategorie

Jede Buchung soll technisch einer Kategorie zugeordnet werden können. Deshalb wird die Auffangkategorie „Nicht kategorisiert“ angelegt. In der Oberfläche erscheint sie als „Ohne Kategorie“ und kann nicht wie eine normale Kategorie gelöscht werden.

5. Herausforderungen während der Entwicklung

5.1 Kategorien und Schlagwörter

Die Kategorienverwaltung war die größte technische Herausforderung. Kategorien und Schlagwörter mussten angelegt, bearbeitet und gelöscht werden können, gleichzeitig sollte die automatische Zuordnung über Empfänger und Verwendungszweck zuverlässig funktionieren. Die Darstellung wurde mehrfach überarbeitet, weil sich die Karten beim Bearbeiten immer wieder verschoben und die Seite dadurch unübersichtlich wurde.

5.2 Gemeinsame Arbeitszeiten

Passende Termine für die Zusammenarbeit zu finden, war nicht immer einfach. Wir haben uns deshalb regelmäßig, während wir für Klausuren gelernt haben, zusammengesetzt, den aktuellen Stand besprochen und die nächsten Arbeitsschritte abgestimmt.

5.3 Parallele Klausuren und Abgaben

Der Projektzeitraum lag gleichzeitig in der Prüfungsphase. Deshalb mussten größere Funktionen zuerst umgesetzt und kleinere Designverbesserungen sowie die Dokumentation auf die letzte Projektphase verschoben werden.

5.4 Unterschiedliche Vorstellungen zum Design

Bei der Gestaltung gab es teilweise unterschiedliche Vorstellungen zur Anordnung von Buttons, zu Formularen und zur Übersichtlichkeit einzelner Seiten. Am Ende wurde es das Design mit den roten Akzenten, inspiriert von den klassischen Anwendungen der Sparkasse. Wir verbinden Finanzen und Online-Banking mit der roten Farbe und haben uns dann endgültig dafür entschieden.

5.5 Einheitliche Begriffe

Im Laufe des Projekts wurden die Begriffe Benutzerkonto, Finanzkonto, Buchung und Bankinstitut vereinheitlicht. Der Begriff Unterkonto wurde entfernt, weil alle Finanzkonten unabhängig voneinander verwaltet werden und keine Verbindung zu echten Bankkonten besteht.

5.6 Langsame Arbeit mit pgAdmin unter macOS

pgAdmin reagierte auf dem verwendeten Mac sehr langsam. Das Öffnen von Tabellen und das Wechseln zwischen Ansichten dauerte länger als erwartet. Für kleinere Prüfungen wurden deshalb zusätzlich Terminalbefehle genutzt, der abschließende SQL-Dump wurde weiterhin über pgAdmin erstellt.

5.7 UML-Diagramm

Eine erste UML Skizze wurde bereits vor der Programmierung begonnen. Nach Abschluss der Programmierung wurden mit Pyreverse ein Klassen- und ein Paketdiagramm direkt aus den vorhandenen Python-Dateien erzeugt.

6. Funktionstest

Nach Abschluss der Programmierung wurde die Anwendung noch ein letztes Mal vollständig getestet. Dafür wurden mehrere Nutzer und Finanzkonten verwendet, sodass auch Gruppen, Nachrichten und Zugriffsrechte geprüft werden konnten. Nach den letzten Korrekturen des Designs wurden die betroffenen Abläufe erneut ausgeführt und der Code anschließend nicht mehr verändert.

Bereich	Prüfung
Registrierung / Login	Gültige und ungültige Eingaben, Login, Logout
Profil	Profildaten und Passwort ändern
Finanzkonten	Erstellen, umbenennen und löschen
Buchungen	Einnahmen/Ausgaben, Datum, Filter und Sortierung
Kategorien	Kategorien/Schlagwörter verwalten und automatisch zuordnen
Excel	Beispieldatei importieren und Buchungen exportieren
Auswertung	Gesamt, Monat, Jahr und freier Zeitraum
Gruppen / Nachrichten	Einladung, Leseansicht und Nachrichtenaustausch
Zugriffsschutz	Fremde URLs und geschützte Seiten prüfen

7. Entwicklungsverlauf

Projektwoche	Zeitraum	Umgesetzte Schritte
1.	29.06. - 05.07.	29.06.: Aufgabenstellung gemeinsam ausgewertet und den Banking Manager als Projektthema ausgewählt. 30.06.: Pflicht-, Zusatz- und optionale Funktionen gesammelt und erste Aufgaben besprochen. 01.07.: Grundlegendes Datenbankschema mit Nutzern, Finanzkonten, Buchungen und Kategorien geplant. 02.07.: Flask und PostgreSQL lokal eingerichtet und die Datenbankverbindung getestet. 03.07.: Projektstruktur mit Routes, Services und Repositories angelegt. 04.–05.07.: Registrierung und Anmeldung begonnen und erste HTML-Seiten erstellt.
2.	06.07. - 12.07.	06.07.: Registrierung, Login, Logout und Passwortspeicherung weiterbearbeitet. 07.07.: Erstellen und Anzeigen mehrerer Finanzkonten umgesetzt. 08.07.: Einnahmen und Ausgaben sowie die Berechnung der Kontostände ergänzt. 09.07.: Dashboard mit den letzten Buchungen und die einzelne Finanzkontoansicht erstellt. 10.07.: Suche, Filterung und Sortierung der Buchungen begonnen. 11.–12.07.: Erste vollständige Abläufe gemeinsam getestet und kleinere Fehler an Formularen, Navigation und Datenbankabfragen korrigiert.
3.	13.07. - 19.07.	13.07.: Datenbanktabellen und grundlegende Funktionen für Kategorien und Schlagwörter umgesetzt. 14.07.: Erste automatische Zuordnung von Buchungen über Empfänger und Verwendungszweck getestet. 15.–18.07.: Wegen der intensiven Vorbereitung auf eine wichtige und anspruchsvolle Mathematiklausur wurde deutlich weniger am Projekt gearbeitet. In dieser Zeit wurden hauptsächlich kleine Fehler notiert, vorhandene Seiten kurz geprüft und offene Aufgaben gesammelt. 19.07.: Kurze gemeinsame Abstimmung zum aktuellen Stand und Planung der nächsten größeren Arbeitsschritte nach der Klausur.
4.	20.07. - 26.07.	20.–21.07.: Weiterhin nur geringer Projektumfang aufgrund der unmittelbar anstehenden Mathematiklausur. Es wurden vor allem kleinere Korrekturen vorgenommen und die nächsten Aufgaben verteilt.

Projektwoche	Zeitraum	Umgesetzte Schritte
		22.07.: Kategorien und automatische Zuordnung fertiggestellt sowie Excel-Import und -Export ergänzt. 23.07.: Auswertungen sowie Gruppen, Einladungen und Nachrichten weiterentwickelt. 24.07.: Benutzeroberfläche gemeinsam überprüft; Navigation, Formulare, Buttons und Begriffe vereinheitlicht. 25.07.: Kategorienseite überarbeitet, vollständigen manuellen Funktionstest durchgeführt und letzte Fehler behoben. SQL-Dump erstellt. 26.07.: Projektordner bereinigt, Klassen- und Paketdiagramm mit Pyreverse erzeugt und den finalen Code in einem neuen GitHub-Repository gesichert.
5. / Abgabetag	27.07.	Projektdokumentation und Verantwortlichkeitsübersicht abschließend kontrolliert, UML-Diagramm und SQL-Dump in die Abgabestruktur eingefügt und die vollständige ZIP-Datei für die Abgabe vorbereitet.

Die Arbeitsschritte ließen sich nicht immer vollständig voneinander trennen. Gestaltung, Abstimmung und Funktionstests fanden während des gesamten Projekts statt. Besonders im Zeitraum vom 15.07. bis 21.07.2026 war der Arbeitsumfang am Projekt aufgrund der parallelen Vorbereitung auf eine wichtige Mathematiklausur geringer als in den übrigen Projektwochen.

8. Abschluss

Insgesamt konnten wir den Banking Manager erfolgreich umsetzen und alle wesentlichen Funktionen der Aufgabenstellung realisieren. Dabei haben wir unsere Kenntnisse in Flask, PostgreSQL und der Aufteilung in Routes, Services und Repositories deutlich erweitert. Besonders hilfreich war es, die vorherigen Hausaufgaben zu machen, wodurch wir uns die Herausforderung bei der Einrichtung der Entwicklungsumgebung und etc. gespart haben.

Rückblickend hätten wir gerne mehr Zeit in die gemeinsame Arbeit mit Git investiert. Branches, Commits, Pushes und Merges waren für uns neu und hätten eine zusätzliche Einarbeitung erfordert. Wegen parallel laufender Klausuren und Abgaben haben wir stattdessen regelmäßig gemeinsam gearbeitet und den Projektstand direkt abgestimmt. Zum Abschluss wurde der finale Stand in einem neuen GitHub-Repository gesichert. Mit dem Ergebnis sind wir zufrieden und möchten die Anwendung später als Referenzprojekt für unser Portfolio nutzen.